Process & Decision Log: eat-resume

I. Core Design Choices & Architecture

Building the MVP required balancing technical feasibility with the core user need (saving time on manual resume screening).

- **The Two-Agent AI Architecture:** Instead of using one massive AI prompt to do everything, the workflow was split into two specialized Gemini nodes:
    - *Agent 1 (Info Extractor):* Tasked solely with extracting deterministic metadata (Name, Email, Phone).
    - *Agent 2 (Analyzer):* Tasked with complex reasoning, scoring the resume against the Job Description, and determining strengths/gaps.
    - *Why:* This separation of concerns reduced AI hallucinations and ensured that even if the reasoning engine failed, the candidate's contact info was still safely logged.
- **Strict Categorical Routing:** To automate the Gmail drafts, the n8n router needed to know exactly where to send the data.
    - *Decision:* I engineered the AI to separate qualitative data (the "why") from categorical data (the "what"). The AI was forced to output a strict 1-word `status` ("Interview", "Hold", or "Reject") to control the automation routing, while placing its long-form explanation into a separate `recommendation_reasoning` database column.
- **Scope Protection (Frontend Input):** Early testing revealed that allowing mixed file types (Word Docs, RTF) caused downstream parsing errors that broke the pipeline.
    - *Decision:* I restricted the Lovable frontend UI to strictly accept `.pdf` files. This standardization guaranteed system stability for the MVP and ensured the AI always received a predictable data format.

II. Prompt Engineering & Iterations

Getting the AI to behave like a strict, reliable HR assistant required several iterations of prompt design and JSON structuring.

- **Iteration 1: The "Naive" Prompt**
    - *Prompt:* "Evaluate this resume and give a score and recommendation."

    - *Result:* The AI hallucinated dummy data (e.g., "Jane Doe") because it was occasionally receiving raw PDF machine code (`%PDF-1.3`) instead of plain text. When it couldn't read the file, it simply made up the output.
- **Iteration 2: The "Chatty" AI**
    - *Prompt:* "Read this text and tell me if we should interview or reject them."
    - *Result:* The AI returned full sentences (e.g., *"Reject due to unreadable content..."*). This immediately broke the n8n Switch/Router node, which was looking for an exact word match to trigger the email drafts.
- **Iteration 3: The "Master" Prompt (Final)**
    - *Prompt:* "Evaluate this candidate's resume: `{{text}}` Against this specific Job Description: `{{job_description}}`. **CRITICAL INSTRUCTIONS:** For the `status` field, you MUST output EXACTLY ONE WORD: 'Interview', 'Hold', or 'Reject'. For the `reasoning` field, provide a short explanation referencing the Job Description."
    - *Result:* Perfect execution. The AI stopped hallucinating, provided distinct scores based on actual JD mapping, and perfectly triggered the automated email routing.

III. Key Technical Learnings & Roadblocks

- **The "Amnesia" Node Effect (Data Scoping):** In node-based automation (n8n), passing data sequentially can cause earlier data to be erased. When the PDF Extractor node extracted the resume text, it "wiped" the Job Description from the payload. *Learning:* I learned to use global variable mapping (pulling the JD directly from the overarching Loop node) so the AI had access to both the resume and the grading rubric simultaneously.
- **Garbage In, Garbage Out (GIGO):** When the AI gave candidates weird decimal scores (like `0.85`), the initial instinct was that the AI prompt was broken. *Learning:* The root cause was actually the data ingestion. Replacing a faulty community PDF extraction node with a native text-extraction node gave the AI clean English text, instantly fixing its reasoning capabilities.
- **Handling API Rate Limits in Loops:** Once the "closed loop" architecture was successfully built, the system processed resumes *too* fast, triggering a `429 Too Many Requests` error from Google Gemini's free tier. *Learning:*Enterprise automation requires queue management. I learned how to implement "Wait" nodes and "Auto-Retry" logic to act as speed bumps, ensuring the pipeline never crashes during high-volume processing